

BACKEND BLUEPRINT

Verified Community Directory PWA

Secure MVP Backend Architecture

A build-ready technical blueprint for private member data, verified access, trusted devices, shared communication, admin control, and auditability.

MODULAR MONOLITH	PRIVATE DATA	TRUSTED DEVICES	AUDITABLE OPERATIONS
---------------------	-----------------	--------------------	-------------------------

Version 1.0 | Secure MVP Build Blueprint

Prepared for the Community Directory Project

Document Control

A practical blueprint for the backend implementation team, QA, reviewers, and maintainers.

Item	Detail
Document	Backend Blueprint - Secure MVP
Product	Verified Community Directory PWA
Version	1.0
Architecture style	Modular monolith: one deployable FastAPI backend with strongly separated domain modules
Primary purpose	Define backend boundaries, data flows, services, APIs, storage, security controls, deployment, testing, and staged implementation
Companion documents	Product Requirements Document (PRD) and Software Requirements Specification (SRS)
Baseline to replace	Static PWA that directly fetches Google Sheet data, creates client-side Member IDs, and stores messages in localStorage

How to use this document

- Use Sections 1-5 to create the backend repository and its core boundaries before writing feature endpoints.
- Use Sections 6-10 to implement identity, trusted-device, directory, communication, and administrator workflows in the correct order.
- Use Sections 11-14 to configure security, data retention, deployment, monitoring, tests, and operational handover.
- Treat the API contracts and acceptance gates as build checkpoints; do not bypass them merely to make the frontend appear functional.

Core decision

Build a modular monolith first. Do not split this project into microservices. A single FastAPI deployment, one primary database, and clearly separated domain services are simpler to test, cheaper to deploy, and much easier to reason about at the MVP stage.

Table of Contents

A map from architecture to implementation and operations.

1. Backend Mission and Design Principles
 2. Target Architecture and Trust Boundaries
 3. Recommended Backend Stack
 4. Repository Structure and Module Responsibilities
 5. Data Model and Database Collections
 6. Identity, OTP, Session, and Trusted Device Workflows
 7. Directory, Privacy, and Import Workflows
 8. Inbox, Emergency, Media, and Moderation Workflows
 9. API Blueprint and Error Contract
 10. Security Events, Rate Limits, and Audit Design
 11. Background Jobs, Deployment, and Configuration
 12. Testing and Quality Gates
 13. Phased Build Plan and Definition of Done
 14. Operational Runbook and Future Extensions
- Appendix A. Environment Variables
- Appendix B. Backend Build Checklist

1. Backend Mission and Design Principles

The backend is the authority for identity, access, member data, communication, device trust, moderation, and operational records. The frontend is a user interface; it cannot be the security gatekeeper.

1.1 What the backend must solve

Current prototype weakness	Backend response
Public Google Sheet data fetch	Replace with authenticated backend APIs backed by a private database.
Browser-generated Member IDs	Generate random, unique, non-guessable Member IDs server-side after admin approval.
Typed ID treated as proof of identity	Authenticate by registered contact, OTP, passkey, trusted device, and server session.
Inbox/Emergency stored in localStorage	Persist posts centrally with author identity, media references, moderation, and audit history.
No device checks	Use WebAuthn passkeys, device records, secure device cookie, and an admin-approved device-change flow.
No activity visibility	Create security events, flags, admin views, and audit logs for every important action.

1.2 Non-negotiable backend principles

- Backend owns all authorization decisions. The client never chooses its own role, Member ID, author identity, or permitted fields.
- Every protected request validates session status, account status, device status, and required role before returning data or changing state.
- Sensitive personal fields use minimum disclosure. The API returns a projection, not a database dump.
- All security-sensitive actions are revocable: sessions, devices, posts, accounts, and media access can be disabled or removed.
- Every high-impact admin action is auditable with actor, action, target, before/after context, and timestamp.
- No production personal data, secrets, endpoint credentials, OTP values, or fallback directory records live in frontend source code.

1.3 MVP architecture decision

Start simple, stay structured

The backend is a single modular FastAPI application. It contains routes, domain services, repositories, adapters, middleware, and jobs in one codebase. Split into separate services only after actual load, team ownership, or operational needs justify it.

2. Target Architecture and Trust Boundaries

React PWA	HTTPS API Gateway	FastAPI Modular Monolith	MongoDB Private Data	Object Store + SMTP
--------------	----------------------	-----------------------------	-------------------------	------------------------

2.1 Logical component map

Layer	Responsibility	Must never do
Frontend PWA	Render screens, collect input, invoke secure APIs, display server-approved data and statuses.	Trust itself for identity, retain sessions in localStorage, query the database, or expose the full directory.
API / Middleware	Authenticate cookies, validate request schema, authorize route access, apply rate limits, issue correlation IDs.	Contain business logic scattered in endpoints or accept unchecked payloads.
Domain Services	Implement member, OTP, passkey, device, post, media, import, security, and audit rules.	Perform direct UI manipulation or embed cloud credentials in responses.
Repositories	Read/write database records and apply safe query projections.	Own business policy decisions or return raw secret fields.
Adapters	Talk to SMTP, object storage, or other third-party providers behind interfaces.	Leak provider-specific details across all modules.
Database/Object storage	Persist private records and media references.	Be exposed directly to public browsers.

2.2 Trust boundary rules

Boundary	Required rule
Browser -> API	HTTPS only. Cookies are HttpOnly and Secure. Every request is schema-validated and authorized server-side.
API -> Database	Private network credentials only. Database is not publicly accessible from browser clients.
API -> SMTP	Use an adapter with environment-based credentials. OTP values are never stored or logged in plaintext.
API -> Object storage	Generate short-lived signed upload/access URLs only after auth and ownership checks.
Admin -> Admin APIs	Role check, audit record, and validation must occur for every high-impact action.

Frontend change rule

No backend endpoint may accept `author_member_id`, `account_status`, `role`, `device_status`, or `is_admin` values from a user-controlled client as a trusted authority. Derive those values from the verified server context.

3. Recommended Backend Stack

Area	Choice	Why it fits the MVP
Language	Python	Matches the project team's existing familiarity and supports a clean typed backend workflow.
API framework	FastAPI	Strong request validation, automatic API docs, async support, and clear dependency-based authorization.
Validation	Pydantic models	Define strict request/response contracts and reject malformed payloads early.
Database	MongoDB	Works well for member profiles with flexible fields while supporting indexed search, audit records, and TTL expiry.
Database access	Async MongoDB client	Avoid blocking the FastAPI event loop during database operations.
Authentication	Opaque server-side sessions	Easy to revoke per account or device and safer than browser-stored access tokens.
Trusted device	WebAuthn / Passkeys + device cookie	Uses browser-supported public-key authentication rather than unavailable IMEI access.
OTP delivery	Email via SMTP adapter	Reasonable and low-complexity MVP identity channel; SMS can be added later if budget is approved.
Media	Private S3-compatible object storage	Supports private uploads and signed access without public Drive links.
Caching / rate limit	In-memory for pilot; Redis when needed	Keep MVP small, but isolate the rate-limit interface so it can move to Redis without route rewrites.
Deployment	Dockerized FastAPI backend	Repeatable environment and clear separation of secrets from source code.

3.1 Intentional exclusions

- No microservices, Kubernetes, Kafka, or event bus for the first secure MVP.
- No native-app-only assumptions such as IMEI collection.
- No public Google Sheet or Google Drive dependency for live production data.
- No JWT stored in localStorage and no client-side author verification.
- No WebSocket requirement at the start; inbox and emergency feeds use controlled polling.

4. Repository Structure and Module Responsibilities

Recommended backend repository structure

```

backend/
  app/
    main.py          # app factory, middleware, router wiring

```

```

api/
  router.py          # route composition
  auth.py            # identify, OTP, logout, /me
  webauthn.py        # registration and assertion endpoints
  members.py         # directory and profile APIs
  posts.py           # inbox and emergency APIs
  media.py           # upload-intent and access-url APIs
  admin.py           # imports, member/device/admin operations
  health.py          # liveness/readiness
core/
  config.py          # typed environment settings
  security.py         # cookie, hash, crypto helpers
  dependencies.py     # current_session, current_member, require_admin
  errors.py          # standard API error model
  logging.py          # structured logging / correlation id
  rate_limit.py       # limit interface + policy registry
domain/
  auth_service.py
  otp_service.py
  webauthn_service.py
  device_service.py
  member_service.py
  import_service.py
  post_service.py
  media_service.py
  security_service.py
  audit_service.py
repositories/
  members_repo.py
  sessions_repo.py
  devices_repo.py
  credentials_repo.py
  otp_repo.py
  posts_repo.py
  media_repo.py
  security_repo.py
  audit_repo.py
schemas/
  auth.py, members.py, posts.py, admin.py, common.py
jobs/
  cleanup.py          # expiry cleanup, retention checks
  imports.py          # later: controlled import sync
adapters/
  smtp_adapter.py
  object_storage_adapter.py
tests/
  unit/, integration/, e2e/
Dockerfile
pyproject.toml
.env.example
docker-compose.yml

```

4.1 Module contract

Layer	Can depend on	Must not depend on
api	schemas, dependencies, domain services	database driver directly for business operations
domain services	repositories, adapters, core security utilities	HTTP request/response objects or frontend concepts
repositories	database client and data models	SMTP, object storage, or role policy

Layer	Can depend on	Must not depend on
		decisions
adapters	provider SDK/client and typed input	route-specific presentation rules
jobs	domain services	frontend endpoints or raw credentials in job payloads

4.2 Endpoint responsibilities

Routes should be thin. A route validates the request schema, reads the authenticated context, calls a service, and returns a typed response. A route should not manually create member IDs, compose OTP logic, parse CSV details, decide privacy fields, or write audit events itself.

5. Data Model and Database Collections

5.1 Core collections

Collection	Primary purpose	Key fields
members	Approved member profile, status, privacy settings, role and secure Member ID.	member_id, registered_email_normalized, status, role, profile, visibility_settings
devices	Server-side trusted-device record and revocation state.	member_id, device_cookie_hash, status, credential_id, approved_at, revoked_at
webauthn_credentials	Passkey public-key material and signature counter.	credential_id, member_id, device_id, public_key, sign_count
sessions	Opaque browser sessions linked to a member and device.	session_token_hash, member_id, device_id, expires_at, revoked_at
otp_challenges	Short-lived OTP challenge state.	purpose, otp_hash, attempts, resend_after, expires_at, consumed_at
device_change_requests	Pending requests to register a new device.	member_id, status, requested_at, reviewed_by, review_reason
posts	Inbox and Emergency content with moderation/resolution state.	type, author_member_id, message, media_ids, status, created_at
media_objects	Private stored media metadata.	owner_member_id, storage_key, content_type, size_bytes, status
security_events	Low-level suspicious or failed events.	event_type, member_ref, source_hash, metadata, created_at
security_flags	Admin-reviewable aggregated security findings.	rule_code, severity, target_ref, status, evidence_event_ids
audit_logs	High-impact administrative and system action trail.	actor, action, target, before, after, created_at
import_runs	CSV import metadata, row validation result, and approval trail.	uploader_id, source_name, status, totals, created_at

5.2 Recommended indexes

Database index plan

```

members.member_id          UNIQUE
members.registered_email_normalized  UNIQUE
webauthn_credentials.credential_id  UNIQUE
devices.device_cookie_hash  UNIQUE
sessions.session_token_hash  UNIQUE
sessions.expires_at        TTL
otp_challenges.expires_at   TTL
posts.type + posts.created_at  COMPOUND DESC
posts.author_member_id + posts.created_at  COMPOUND DESC
security_events.created_at    DESC
security_flags.status + severity  COMPOUND
import_runs.created_at        DESC

```

5.3 State machines

Member lifecycle

STAGED	PENDING ENROLLMENT	ACTIVE	SUSPENDED	DEACTIVATED
--------	-----------------------	--------	-----------	-------------

Device lifecycle

PENDING	ACTIVE	REVOKED
---------	--------	---------

Emergency lifecycle

ACTIVE	RESOLVED	ARCHIVED
--------	----------	----------

Data rule

Store only hashes of browser session and trusted-device token values. Raw cookie values are sent once to the browser and must never be stored, exported, or logged in recoverable form.

6. Identity, OTP, Session, and Trusted Device Workflows

6.1 First-time enrollment

Member ID + registered email	Generic identify result	OTP verify	Passkey register	Device ACTIVE	Session created
---------------------------------	----------------------------	---------------	---------------------	------------------	--------------------

Step	Backend behavior
1. Identify	Accept Member ID and email. Normalize inputs. Return a generic message whether details match or not.

Step	Backend behavior
2. OTP issue	When an approved pending member matches, create a hashed OTP challenge with expiry, resend cooldown, and attempt limit. Dispatch email through adapter.
3. OTP verify	Validate challenge purpose, expiry, attempt count, OTP hash, and request limits. Mark challenge consumed on success.
4. Enrollment session	Create a short-lived server-side enrollment context limited to WebAuthn registration.
5. Passkey registration	Issue WebAuthn options; verify challenge, origin, relying party, credential ID, public key, and user verification requirement.
6. Device activation	Create ACTIVE device record, hash/store trusted-device cookie, issue opaque member session, set account ACTIVE.

6.2 Returning approved device login

Device cookie present	Passkey assertion	Server verify	Session issue	Protected access
-----------------------	-------------------	---------------	---------------	------------------

- The backend validates the WebAuthn assertion against the stored credential, current challenge, expected origin, expected relying-party ID, and signature counter.
- The backend validates that the trusted-device cookie hash maps to the same ACTIVE device and approved member.
- The backend validates that the member status is ACTIVE and has not been suspended or deactivated.
- Only then is a fresh opaque session issued. The session is linked to the active device record.

6.3 New or reset device flow

No active device cookie	OTP verification	Device change request	Admin approval	New passkey register	Old device revoked
-------------------------	------------------	-----------------------	----------------	----------------------	--------------------

Mandatory policy

OTP proves control of the registered email. It does not, by itself, bypass the trusted-device policy. A new browser, cleared-cookie browser, or new phone remains blocked from protected data until an administrator approves the device change and a new passkey/device record is created.

6.4 Cookie strategy

Cookie	Purpose	Required attributes
__Host-cd_session	Opaque authenticated session reference.	HttpOnly, Secure, Path=/, SameSite=Strict, short expiry
__Host-cd_device	Opaque trusted-device reference.	HttpOnly, Secure, Path=/, SameSite=Strict, long-lived but revocable
No auth token in localStorage	Protect session material from direct	Never use localStorage/sessionStorage for

Cookie	Purpose	Required attributes
	JavaScript access.	credential tokens

7. Directory, Privacy, and Import Workflows

7.1 Secure import pipeline

Google Form / CSV export	Admin upload	Validation + duplicate checks	STAGED records	Admin approval	Secure ID + enrollment
--------------------------	--------------	-------------------------------	----------------	----------------	------------------------

Import stage	Required backend action
Upload	Allow only Admin role. Capture source name, uploader, timestamp, and checksum/metadata.
Parse	Validate CSV structure, required columns, normalise email/phone fields, and reject unsafe or malformed rows.
Detect conflicts	Flag duplicate email, duplicate phone, duplicate Member ID attempt, and likely duplicated identity for review.
Stage	Store valid rows as STAGED. Do not expose them to member directory APIs.
Approve	Admin explicitly approves. Backend generates random Member ID and sets PENDING_ENROLLMENT.
Audit	Record import action, row summary, decision owner, and changes.

7.2 Directory query contract

Directory API behavior

GET /api/v1/members
 ?q=<search text>
 &state=<value>
 &blood_group=<value>
 &occupation=<value>
 &education_sector=<value>
 &page=1&page_size=20

Response must include only directory-card fields:
 member_id, name, profile_photo_access, city, state,
 occupation, organisation, permitted badges, updated_at

GET /api/v1/members/{member_id}
 returns a privacy-filtered profile projection for the current viewer.

7.3 Privacy projection service

Field category	Default projection
Directory card	Name, photo/avatar, city/state, occupation/organisation where approved.
Member profile	Only fields permitted to verified members by profile setting and policy.
Sensitive personal information	Admin-only by default: date of birth, detailed family fields, child/spouse data, pincode, security history.
Contact details	Visible only when the member setting and community policy allow it.
Caste/sub-caste	Admin-only by default. Member-facing display requires explicit policy decision and member consent.
Security/device/session fields	Never returned to normal member profile endpoints.

Projection rule

Do not write one giant member serializer and reuse it everywhere. Create explicit response schemas for directory cards, self-profile, member-profile-for-member, and admin profile views.

8. Inbox, Emergency, Media, and Moderation Workflows

8.1 Unified post model

For the MVP, use one posts collection. Use a type field to distinguish INBOX and EMERGENCY records. The service layer applies different status, display, and resolution rules by type.

Posts collection schema concept

```
posts document (simplified)
{
  _id,
  type: "INBOX" | "EMERGENCY",
  author_member_id: <derived from session>,
  message: <validated text>,
  media_ids: [<private media ids>],
  status: "ACTIVE" | "REMOVED" | "RESOLVED" | "ARCHIVED",
  priority: "NORMAL" | "URGENT",
  reported_count,
  moderation: { reason, actioned_by, actioned_at },
  resolution: { note, resolved_by, resolved_at },
  created_at, updated_at
}
```

8.2 Message and alert creation

Verified member	POST content	Session-derived author	Validation + media check	Central persist	Shared feed
-----------------	--------------	------------------------	--------------------------	-----------------	-------------

- The request body never contains an authoritative `author_member_id`. The server derives author identity from the current session.
- Validate text size, blank content rules, allowed media references, and member/device status before saving.
- Emergency records receive URGENT priority and can be marked RESOLVED by an Admin with a resolution note.
- Feeds use pagination and controlled polling in MVP. Do not add WebSockets until there is a demonstrated need.
- Members may report posts. Reports become security/moderation input and do not immediately remove content automatically.

8.3 Private media flow

Verified member	Upload intent	Signed upload URL	Storage confirmation	Media attach to post	Signed read URL
-----------------	---------------	-------------------	----------------------	----------------------	-----------------

Control	MVP policy
Allowed content	Images and videos only. Reject unsupported MIME types.
Size limits	Images up to 5 MB; videos up to 20 MB. Tune after pilot feedback.
Ownership	A media object can be attached only by its verified owner or by an authorized administrator.
Visibility	Media remains private; provide short-lived access URLs only for viewers allowed to see the associated post/profile.
Moderation	If post is removed, associated media access must be revoked or blocked according to retention policy.

8.4 Moderation and resolution

Admin action	Backend behavior
Remove Inbox post	Soft-remove content from normal feed, preserve moderation metadata and audit record.
Resolve Emergency	Set status to RESOLVED, capture resolver, timestamp, and optional note; retain alert history.
Suspend member	Revoke active sessions, prevent new posts, preserve historical authorship for accountability.
Revoke device	Mark device revoked and invalidate all sessions tied to it.

9. API Blueprint and Error Contract

9.1 Route groups

Route group	Core endpoints	Access
Health	GET /health/live, GET /health/ready	Public infrastructure checks only
Auth	POST /auth/identify, POST /auth/otp/verify, POST /auth/logout, GET /auth/me	Public initiation; protected session reads
WebAuthn	POST /webauthn/register/options, POST /webauthn/register/verify, POST /webauthn/login/options, POST /webauthn/login/verify	Enrollment or device-auth context only
Devices	GET /devices/current, POST /devices/change-request, DELETE /devices/current	Authenticated member
Members	GET /members, GET /members/{member_id}, GET /members/filters	Authenticated active member/device
Posts	GET /posts, POST /posts, POST /posts/{id}/report	Authenticated active member/device
Emergency	GET /emergency-alerts, POST /emergency-alerts, POST /emergency-alerts/{id}/resolve	Member read/post; Admin resolve
Media	POST /media/upload-intent, POST /media/complete, GET /media/{id}/access-url	Authenticated owner/viewer
Admin	Imports, approvals, member management, flags, moderation, exports	Admin only

9.2 Response envelope

API response convention

Success response

```
{
  "data": { ... },
  "meta": { "request_id": "...", "page": 1, "page_size": 20 }
}
```

Error response

```
{
  "error": {
    "code": "AUTHENTICATION_REQUIRED",
    "message": "Please sign in again.",
    "request_id": "..."
  }
}
```

9.3 Error rules

Condition	External response	Internal handling
Invalid identity details	Generic success-style message that avoids account enumeration.	Record security event with safe metadata.

Condition	External response	Internal handling
Expired OTP	Verification code expired. Request a new code.	Do not expose stored challenge details.
Invalid session	Please sign in again.	Clear/revoke when applicable and log safely.
Device not approved	Device approval is required.	Create/preserve device-change request state.
Permission denied	You do not have permission for this action.	Record access decision where appropriate.
Validation failure	Show field-specific safe error message.	No database mutation.
Unexpected failure	Unable to complete the request right now.	Structured error log with request_id; no stack trace to client.

API safety rule

Use versioned APIs from the start: /api/v1/.... Never expose stack traces, MongoDB errors, raw provider error objects, or security-token details to the frontend.

10. Security Events, Rate Limits, and Audit Design

10.1 Security event taxonomy

Event type	Typical trigger	Recommended response
IDENTITY_MISMATCH	Member ID + registered email do not match an approved record.	Generic response, rate-limit source if repeated.
OTP_FAILED	Incorrect OTP attempt.	Increment challenge count; expire after threshold.
OTP_RESEND_ABUSE	Rapid repeated resend attempts.	Cooldown and temporary source limit.
DEVICE_TOKEN_INVALID	Unknown/revoked trusted-device cookie.	Reject; flag if repeated.
WEBAUTHN_ASSERTION_FAILED	Invalid challenge/signature/origin/credential state.	Reject; log safe metadata; flag if repeated.
UNAUTHORIZED_POST_ATTEMPT	Post without valid session, active device, or status.	Reject and record.
MEDIA_REJECTED	Unsupported file type, oversize file, or ownership mismatch.	Reject; record if suspicious.
ADMIN_ACTION	High-impact dashboard change.	Always create audit record.

10.2 Initial rate-limit policy

Action	Initial MVP policy	Purpose
Identity initiation	5 attempts per source context per 15 minutes	Reduce enumeration and brute force.

Action	Initial MVP policy	Purpose
OTP verification	5 attempts per OTP challenge	Expire challenge safely without broad account lockout.
OTP resend	1 resend per 60 seconds; daily cap per member/source	Prevent email abuse.
Passkey login options	Short rolling limit per source/device	Reduce challenge spam.
Post creation	Reasonable per-member burst limit	Reduce automated spam.
Media upload intent	Limit concurrent uploads per member	Control abuse and costs.

10.3 Security flag aggregation

Security event	Rule evaluation	Security flag	Admin review	Resolution audit
----------------	-----------------	---------------	--------------	------------------

Do not auto-lock an unrelated real member account because an attacker repeatedly enters their visible Member ID incorrectly. Limit the requesting source or challenge first. Suspend member/device only after verified suspicious behavior or administrator action.

10.4 Audit log requirements

- Audit every approval, suspension, activation, profile edit, device decision, content moderation, emergency resolution, export, and role change.
- Each audit entry records actor, action, target type/ID, timestamp, reason if applicable, and safe before/after snapshot.
- Audit logs are append-only from the application perspective. Normal members cannot view them.
- Do not put OTPs, raw cookies, passwords, passkey private material, or unredacted secrets into audit records.

11. Background Jobs, Deployment, and Configuration

11.1 MVP background jobs

Job	Trigger	Action
OTP email delivery	After challenge creation	Use a background task for dispatch; OTP state is stored before sending.
Expired session / OTP cleanup	Scheduled / TTL index	TTL removes expired data; job may also log cleanup summaries.
Security-flag evaluation	After security event	Evaluate rule thresholds and create/update flag in same service flow or short background task.
Media orphan cleanup	Scheduled	Delete/mark media not attached to any valid record after grace period.
Import report generation	After CSV parse	Create summary and validation errors for admin review.
Later: Sheet sync	Admin-approved scheduled job	Import to staging only. Never publish raw

Job	Trigger	Action
		Sheet data directly to members.

11.2 Deployment topology

Browser PWA	HTTPS frontend host	FastAPI container	MongoDB private cluster	SMTP + object store
----------------	------------------------	----------------------	----------------------------	------------------------

Deployment component	Minimum responsibility
Frontend host	Serve static React/PWA assets over HTTPS. Configure only public API base URL, not secrets.
Backend container	Run FastAPI with production server, health checks, environment configuration, and structured logging.
Database	Private access, backups, network restrictions, indexes, and retention policy.
Object storage	Private bucket, signed URL policy, lifecycle/retention settings.
SMTP provider	Dedicated transactional sender, domain/account verification, rate limits, delivery monitoring.

11.3 Environment configuration

Environment variables - illustrative baseline

```

APP_MODE=development|production
APP_ORIGIN=https://app.example.org
API_ORIGIN=https://api.example.org
MONGODB_URI=<private connection string>
DATABASE_NAME=community_directory
JWT_SECRET_KEY=<only if used for signed short-lived internal state>
SESSION_SECRET_KEY=<required>
DEVICE_TOKEN_SECRET_KEY=<required>
WEBAUTHN_RP_ID=app.example.org
WEBAUTHN_RP_NAME=Community Directory
SMTP_HOST=<smtp host>
SMTP_PORT=<port>
SMTP_USERNAME=<sender account>
SMTP_PASSWORD=<secret>
OTP_FROM_EMAIL=<approved sender>
OBJECT_STORAGE_ENDPOINT=<private>
OBJECT_STORAGE_BUCKET=<private bucket>
OBJECT_STORAGE_ACCESS_KEY=<secret>
OBJECT_STORAGE_SECRET_KEY=<secret>
ADMIN_INIT_USERNAME=<required first boot>
ADMIN_INIT_PASSWORD=<required first boot>
CORS_ALLOWED_ORIGINS=https://app.example.org

```

Secret handling

Use environment variables or a managed secret store. Do not commit .env files, passwords, cloud keys, bootstrap credentials, or mail tokens to Git. Production startup must fail clearly when mandatory secrets are missing.

12. Testing and Quality Gates

12.1 Test layers

Layer	What it validates	Examples
Unit tests	Pure domain rules and utilities.	Member ID generation, OTP hash verification, rate-limit rule evaluation, privacy projection.
Repository tests	Database persistence and indexes.	TTL fields, unique ID conflicts, query projections, audit record creation.
API integration tests	Route + auth + validation + service composition.	Unauthenticated directory denied, admin approval succeeds, bad OTP increments count.
WebAuthn tests	Registration/login challenge lifecycle and credential binding.	Wrong origin rejected, revoked device denied, sign counter update handled.
End-to-end tests	Frontend-to-backend journeys.	First enrollment, old-device revoke, post visible to second verified member.
Security regression tests	Known bypass attempts.	Spoofed author_member_id ignored, localStorage token absent, direct CSV endpoint unavailable.

12.2 Minimum acceptance gates before pilot

- Unauthenticated users receive no directory, profile, post, emergency, or media content.
- No public Google Sheet fetch or real member fallback dataset remains in frontend production build.
- A post author is always derived from the verified session, even if the request tries to send another author ID.
- An OTP cannot be reused, outlive its expiry, or be brute-forced beyond the configured challenge limit.
- A new device cannot access protected APIs after OTP only; it requires admin approval and new passkey/device activation.
- Revoking a device invalidates sessions attached to that device.
- Admins can view, moderate, and audit Inbox/Emergency activity.
- Security events and admin actions contain request identifiers but never raw secrets.
- PWA cache contains only safe app-shell assets, not private profile feed payloads.

12.3 Definition of done for any backend feature

Check	Required evidence
Contract	Pydantic request/response schema and endpoint behavior documented.
Authorization	Route and service enforce correct session/device/role checks.
Validation	Malformed or unauthorized input is rejected safely.
Persistence	Database writes are reliable and indexes/constraints are

Check	Required evidence
	appropriate.
Audit/security	High-impact action creates appropriate audit/security event.
Tests	Unit + integration tests cover happy path and key failure path.
Observability	Errors include request_id and no sensitive values are logged.

13. Phased Build Plan and Definition of Done

Phase	Backend deliverable	Exit gate
0. Legacy freeze	Move existing static prototype to UI-reference folder; remove real data from source fixtures.	No production personal data, Google Sheet ID, or public form URL in frontend source.
1. Foundation	FastAPI app, async DB connection, config, health checks, error contract, admin bootstrap, audit utility.	Backend boots in Docker and readiness endpoint confirms private DB access.
2. Admin + import	Admin auth, CSV import, staging, validation, approval, secure Member IDs.	Admin can import/approve members without public directory exposure.
3. Protected directory	Session guard, member query/profile APIs, filters, privacy projection.	Verified active session/device is required for every directory route.
4. OTP + sessions	Identify, OTP issue/verify, cookie session lifecycle, security events.	Member cannot reach directory by merely knowing an ID.
5. Trusted devices	WebAuthn registration/login, device record, request/approval/revoke flow.	New device needs admin approval; revoked device loses access.
6. Shared communication	Inbox/Emergency posts, media, reporting, moderation, resolution.	Post from one member is central, visible to others, and auditable.
7. Security operations	Flags, dashboards, exports, cleanup jobs, pilot monitoring.	Admins can investigate operational and security events.
8. Pilot hardening	E2E tests, safe PWA caching, backup/restore check, browser/device matrix.	Ready for limited real-member pilot.

13.1 Build order rule

Do not invert the sequence

Do not build passkeys, media uploads, or an elaborate admin dashboard before member records, sessions, and protected APIs exist. Establish a private database and backend source of truth first; then layer identity, devices, communication, and operations on top.

14. Operational Runbook and Future Extensions

14.1 Minimum operational runbook

Situation	Immediate action	Follow-up
Member cannot access on old approved device	Check account, device, and session status; confirm passkey browser support.	Review security events; revoke/re-enroll only if necessary.
Member changed phone/browser	Guide through OTP and Device Change Request; do not bypass policy manually without audit.	Admin approves/rejects; old device sessions are revoked on approval.
Suspicious OTP activity	Apply rate limit and inspect events/source context.	Create/resolve security flag; notify relevant admin if needed.
Inappropriate post or media	Remove/soft-hide through Admin Dashboard with reason.	Retain audit entry and consider member/device action if repeated.
Emergency alert resolved	Admin marks resolved with note.	Keep history for review; do not delete active audit trail.
Import contains duplicate or malformed data	Keep records in STAGED/rejected state.	Resolve data with community administrator; never auto-activate.
Backend database unavailable	Fail closed for protected operations; show safe unavailable message.	Investigate health checks, restore connectivity, verify no partial writes.

14.2 Future extensions after secure MVP

- Optional SMS OTP channel with approved provider, budget, delivery monitoring, and policy review.
- Push notifications for emergency alerts after consent, supported browser strategy, and operational ownership are defined.
- Controlled Google Sheet sync job into staging, not live public access.
- Search enhancements, saved filters, volunteer coordination, event calendar, and targeted announcements.
- Real-time transport only after feed polling proves insufficient and monitoring capacity exists.
- Data-retention automation, privacy self-service, and advanced reporting after community governance policy is finalized.

Appendix A. Environment Variable Checklist

Category	Required variables
Runtime	APP_MODE, APP_ORIGIN, API_ORIGIN, LOG_LEVEL
Database	MONGODB_URI, DATABASE_NAME
Security	SESSION_SECRET_KEY, DEVICE_TOKEN_SECRET_KEY, COOKIE_DOMAIN where applicable
WebAuthn	WEBAUTHN_RP_ID, WEBAUTHN_RP_NAME, WEBAUTHN_ORIGINS
OTP/Email	SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, OTP_FROM_EMAIL
Object storage	OBJECT_STORAGE_ENDPOINT, OBJECT_STORAGE_BUCKET, OBJECT_STORAGE_ACCESS_KEY,

Category	Required variables
	OBJECT_STORAGE_SECRET_KEY
Bootstrap	ADMIN_INIT_USERNAME, ADMIN_INIT_PASSWORD
Frontend control	CORS_ALLOWED_ORIGINS, TRUSTED_PROXY_HEADERS policy

Appendix B. Backend Build Checklist

- Create repo and environment setup; commit .env.example only, never real secrets.
- Implement config validation and production startup failure for missing mandatory security settings.
- Create database clients, indexes, health routes, request IDs, centralized errors, structured logging.
- Implement secure administrator bootstrap and audit service before dashboard features.
- Implement member import/staging/approval/secure ID issuance before directory frontend migration.
- Implement session guard and privacy projections before adding profile endpoints.
- Implement OTP and trusted device lifecycle with integration tests before opening any protected member route.
- Implement post/media/moderation services after identity and access controls are stable.
- Implement flags, reports, cleanup, backups, and pilot monitoring before real-data rollout.